Chương 9: Iterator Pattern và Composite Pattern (P2)

- Composite Pattern là mẫu cho phép kết hợp các đối tượng thành các cấu trúc cây để thể hiện hệ thống phân cấp.
- Composite Pattern cho phép client xử lý các đối tượng đơn và các composite con theo cùng một kiểu.

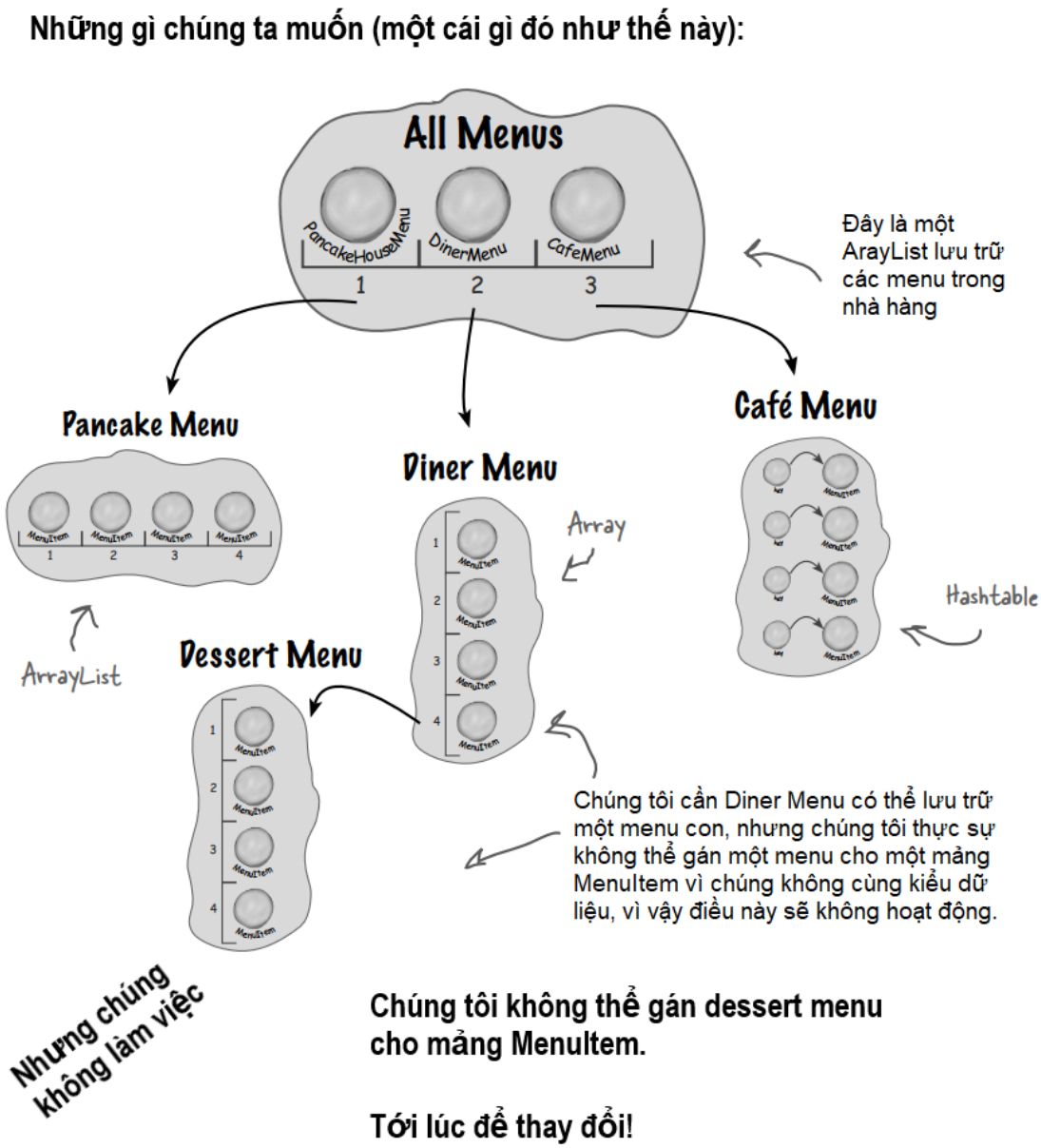
Ngay khi chúng tôi nghĩ rằng nó an toàn...

Bây giờ họ muốn thêm một menu con, món tráng miệng (DessertMenu).



Được rồi, bây giờ thì sao? Bây giờ chúng tôi phải hỗ trợ không chỉ nhiều menu, mà cả các menu con trong menu.

Sẽ thật tuyệt nếu chúng ta có thể biến DessertMenu thành một phần của DinerMenu, nhưng khi đó chúng sẽ không hoạt động giống như code hiện tại.



Chúng ta cần gì?

Đã đến lúc đưa ra quyết định để làm lại thành một thứ đủ chung để làm việc được trên tất cả các menu (cả menu phụ – sub menu). Đúng vậy, chúng tôi sẽ nói với các đầu bếp rằng đã đến lúc chúng tôi phải làm lại thực đơn của họ.

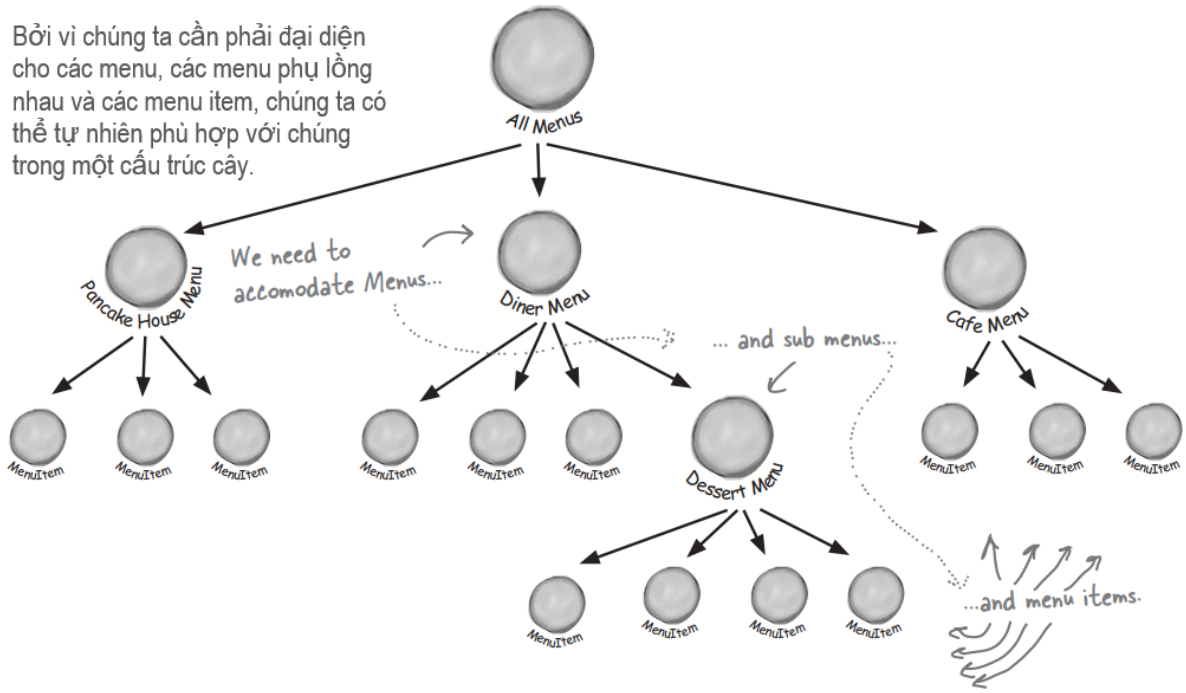
Thực tế là chúng tôi đã đạt đến một mức độ phức tạp đến mức nếu bây giờ chúng tôi không làm lại thiết kế, chúng tôi sẽ không bao giờ có một thiết kế có thể phù hợp với việc kết hợp với **DessertMenu**.

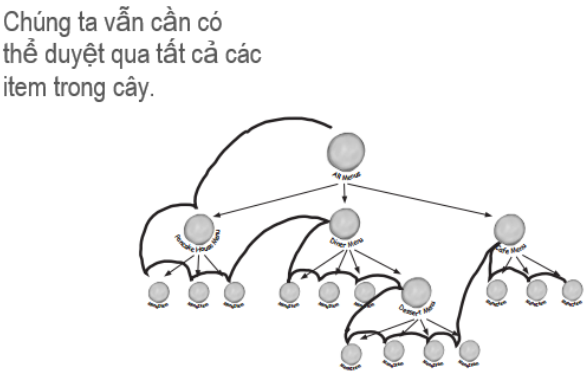
Vì vậy, chúng ta thực sự cần thiết kế mới những gì?

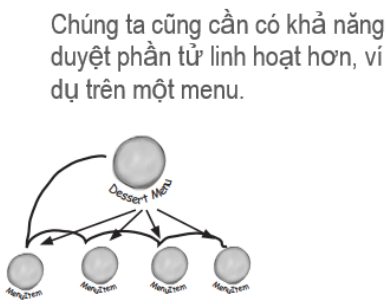
- Chúng ta cần một số loại cấu trúc cây sẽ chứa các menu, menu con và các menu item.
- Chúng ta cần đảm bảo rằng chúng ta có một cách để duyệt qua các phần tử trong mỗi menu ít nhất là thuận tiện như những gì chúng ta đang làm bây giờ với các iterator.
- Chúng ta có thể cần phải duyệt qua các phần tử một cách linh hoạt hơn. Chẳng hạn, chúng ta có thể chỉ cần duyệt trên dessert menu của Diner, hoặc chúng ta có thể cần duyệt toàn bộ Diner menu, bao gồm cả dessert submenu.

Đã đến lúc chúng ta phải cấu trúc lại code của mình để nó phát triển. Không làm như vậy sẽ để lại cho chúng ta code cứng nhắc, không linh hoạt, không có hy vọng sẽ nảy sinh sự sống mới.









Sử dụng sức mạnh bộ não

Bạn sẽ xử lý vấn đề mới này như thế nào theo yêu cầu thiết kế của chúng tôi? Hãy suy nghĩ về nó trước khi đọc tiếp.

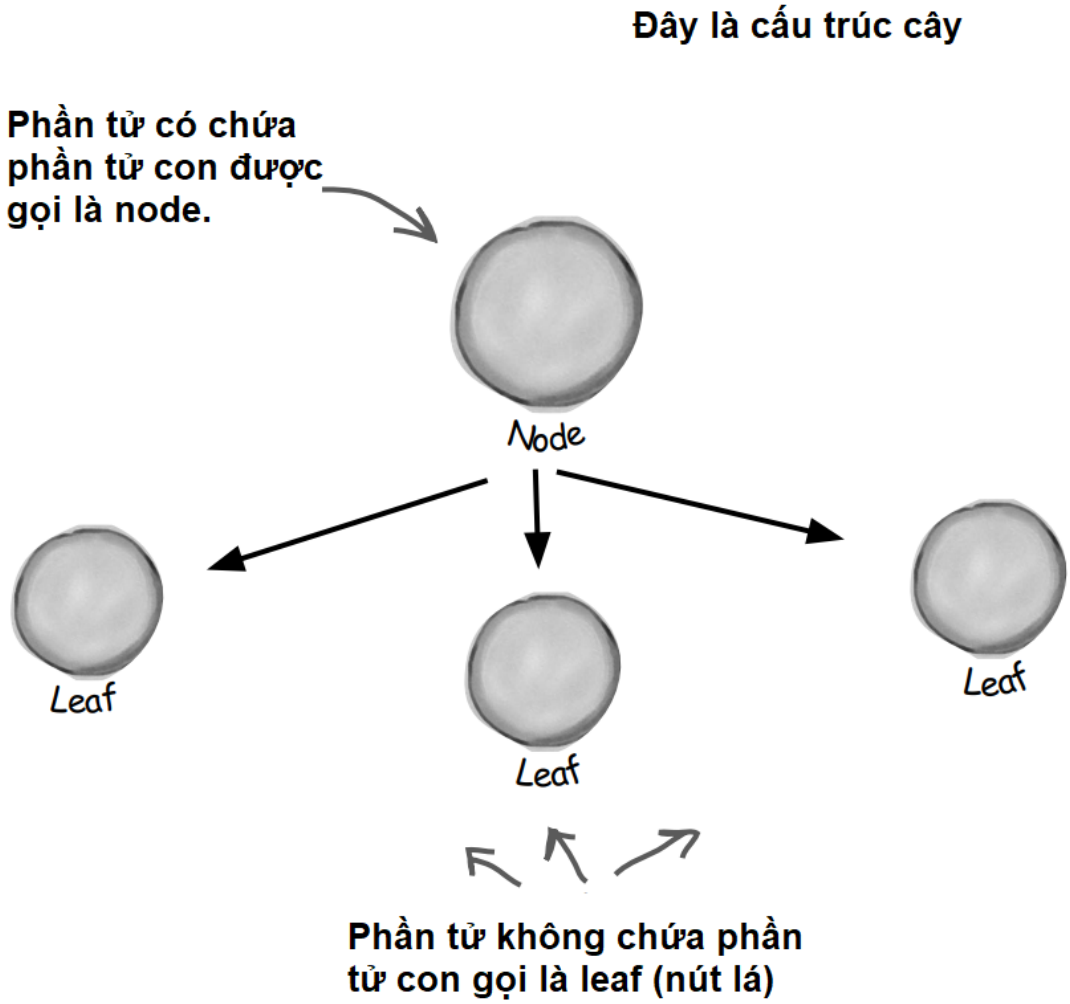
Định nghĩa Composite Pattern

Đúng vậy, chúng tôi sẽ giới thiệu một mẫu thiết kế khác để giải quyết vấn đề này. Chúng tôi đã không từ bỏ Iterator – nó vẫn sẽ là một phần của giải pháp của chúng tôi – tuy nhiên, vấn đề quản lý các menu đã diễn ra ở một khía cạnh mới mà Iterator không giải quyết được. Vì vậy, chúng tôi sẽ lùi lại và giải quyết nó với **Composite Pattern**.

Chúng ta có định nghĩa **Composite Pattern** chính thức bây giờ:

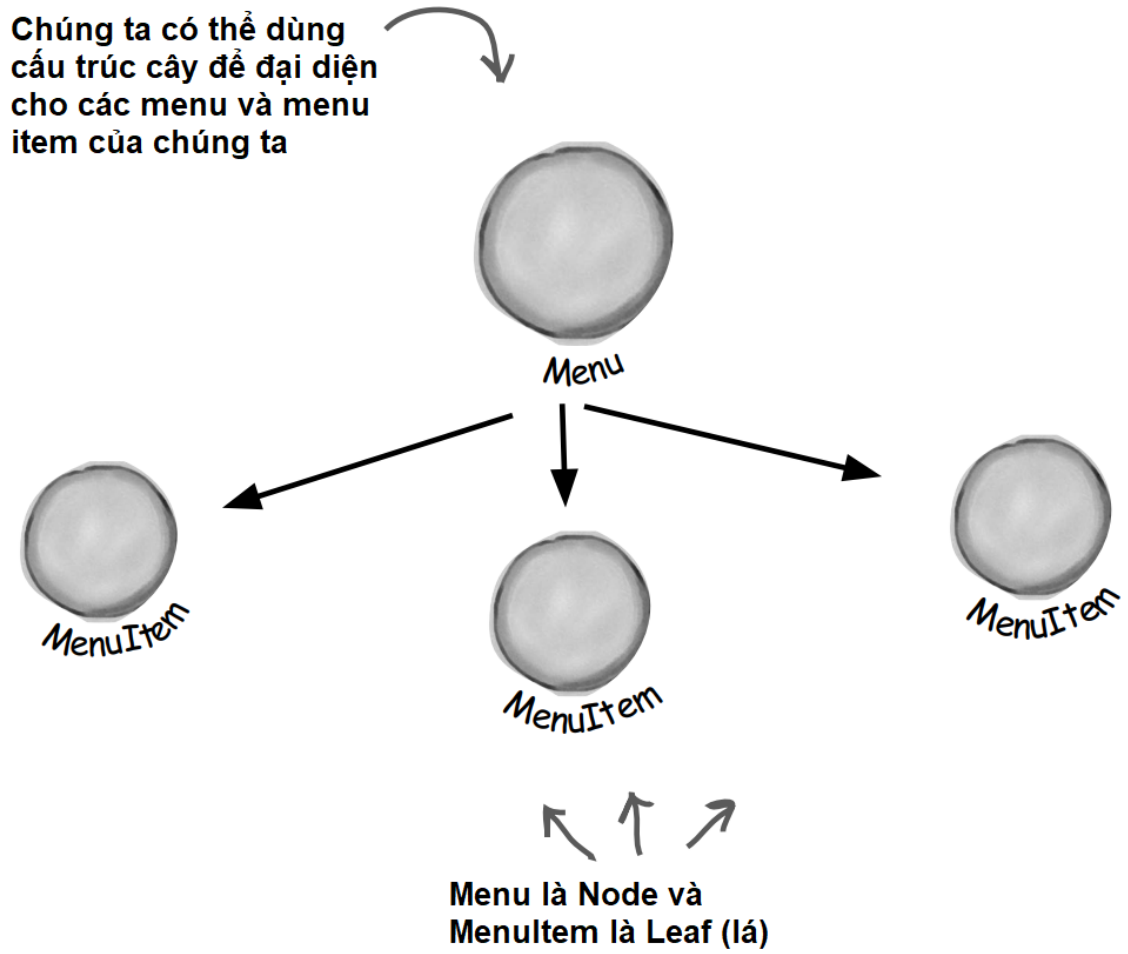
The Composite Pattern allows you to compose objects into tree structures to represent part-whole hierarchies. Composite lets clients treat individual objects and compositions of objects uniformly.

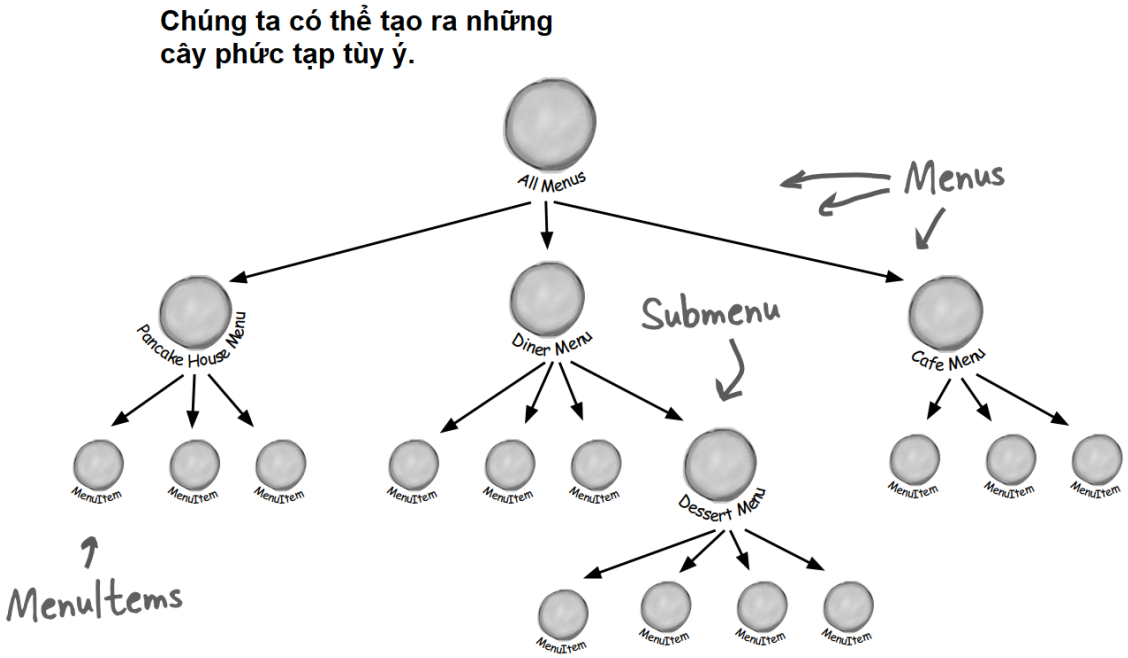
(Composite Pattern cho phép bạn kết hợp các đối tượng thành các cấu trúc cây để thể hiện hệ thống phân cấp toàn bộ. Composite cho phép client xử lý các đối tượng riêng lẻ và các thành phần của các đối tượng một cách thống nhất)



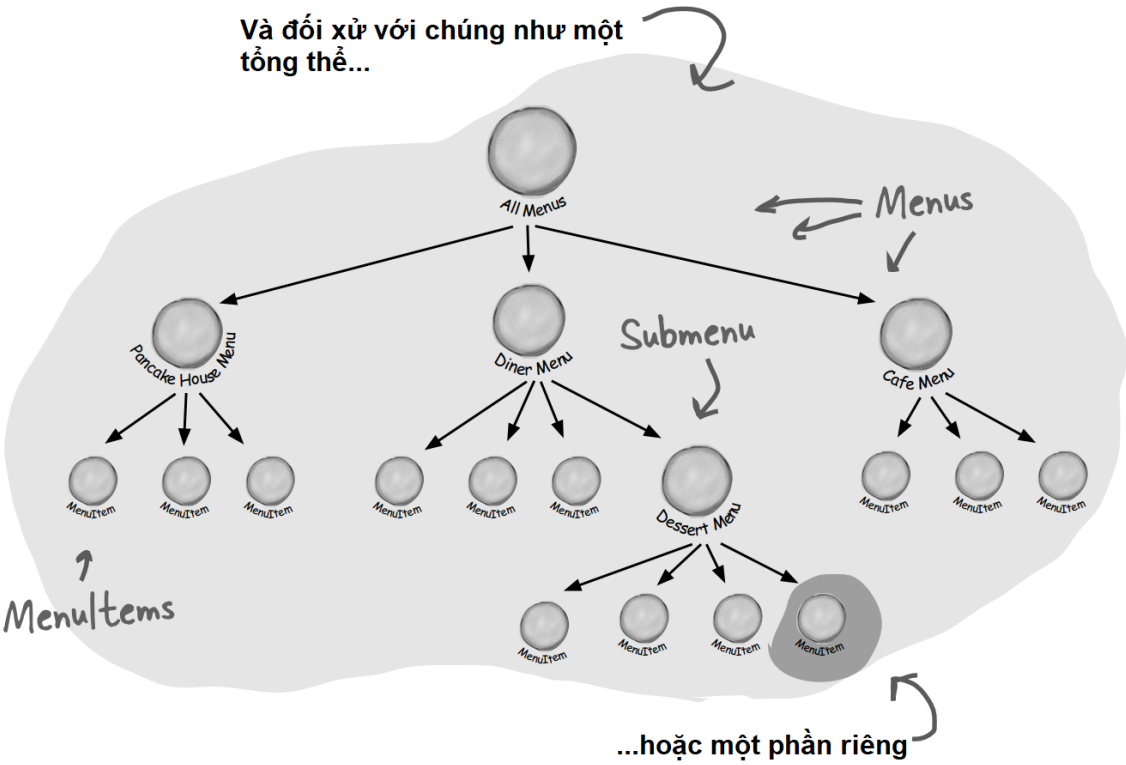
Hãy suy nghĩ về điều này trong các menu của chúng ta: mẫu này cho chúng ta một cách để tạo ra một cấu trúc cây có thể xử lý một nhóm các menu và các menu item lồng nhau trong cùng một cấu trúc. Bằng cách đặt các menu và các item trong cùng một cấu trúc, chúng tôi tạo ra một hệ thống phân cấp toàn bộ; nghĩa là, một cây các đối tượng được tạo thành từ các thành phần (menu và các menu item) nhưng có thể được coi như một tổng thể, giống như một menu lớn.

Khi chúng tôi có menu của mình, chúng tôi có thể sử dụng mẫu này để xử lý thống nhất các đối tượng và thành phần riêng lẻ. Điều đó có nghĩa là gì? Điều đó có nghĩa là nếu chúng ta có một cấu trúc cây gồm các menu, menu con và có lẽ là các menu con của menu con cùng với các menu item, khi đó bất kỳ menu nào cũng là một "composition" vì nó có thể chứa cả các menu khác và các menu item. Các đối tượng riêng lẻ chỉ là các menu item – chúng không lưu giữ các đối tượng khác. Như bạn thấy, sử dụng một thiết kế tuân theo Composite Pattern sẽ cho phép chúng ta viết một số code đơn giản có thể áp dụng cùng một thao tác (như in!) Trên toàn bộ cấu trúc menu.

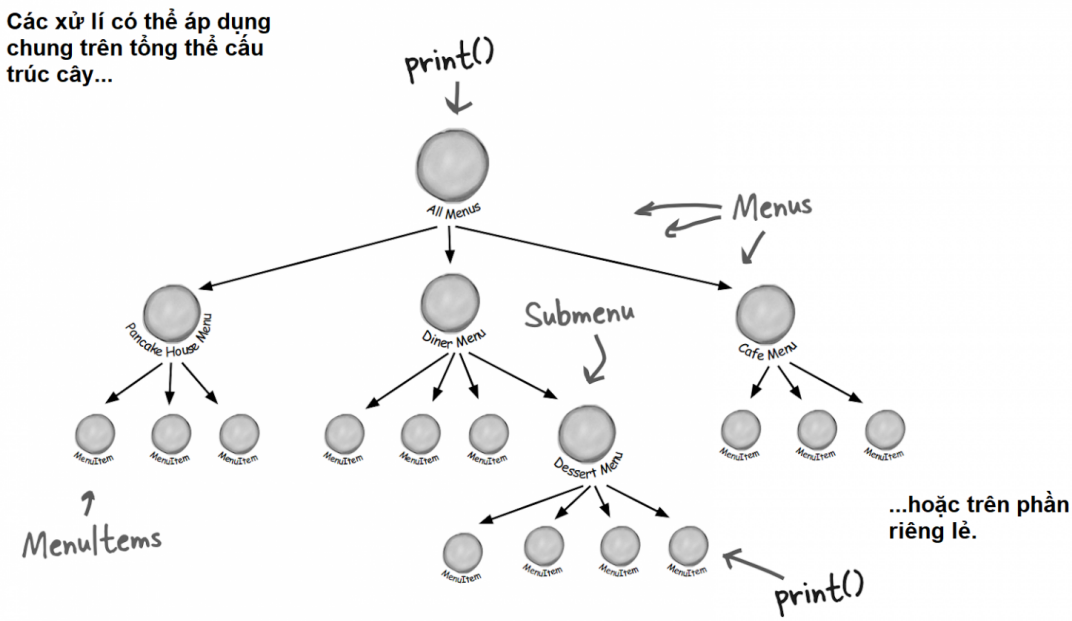


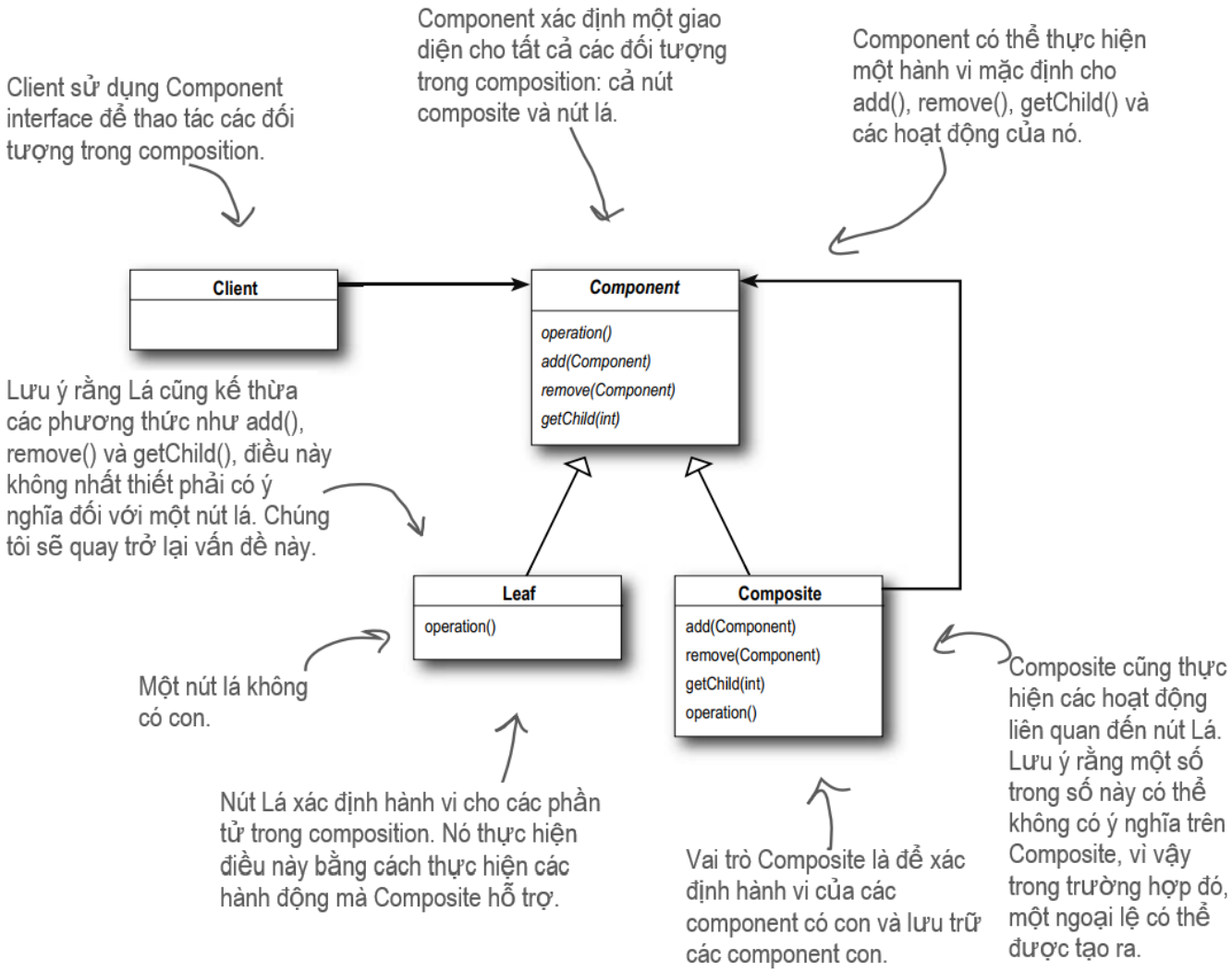


Composite Pattern cho phép chúng ta xây dựng các cấu trúc của các đối tượng ở dạng cây có chứa cả các tập hợp của các đối tượng và các đối tượng riêng lẻ dưới dạng các nút.



Sử dụng một cấu trúc composite, chúng ta có thể áp dụng các hoạt động tương tự trên cả đối tượng composite và các đối tượng riêng lẻ. Nói cách khác, trong hầu hết các trường hợp, chúng ta có thể bỏ qua sự khác biệt giữa các composite của các đối tượng và các đối tượng riêng lẻ.





Không có câu hỏi ngớ ngẩn

Hỏi: Component, Composite, Trees? Tôi hơi bối rối.

Đáp: Một Composite chứa các Component. Component chứa hai loại: composite và các nút lá. Nghe như đệ quy? Chính nó. Một composite chứa một tập hợp các nút con, những nút con đó có thể là composite khác hoặc các nút lá.

Khi bạn tổ chức dữ liệu theo cách này, bạn hoàn thành với cấu trúc cây (thực ra là cấu trúc cây lộn ngược) với một composite ở gốc và các nhánh của composite mọc lên đến các nút lá.

Hỏi: Làm thế nào điều này liên quan đến các iterator (https://toihocdesignpattern.com/chuong-9-iterator-pattern-va-composite-pattern-p1-iterator-pattern.html)?

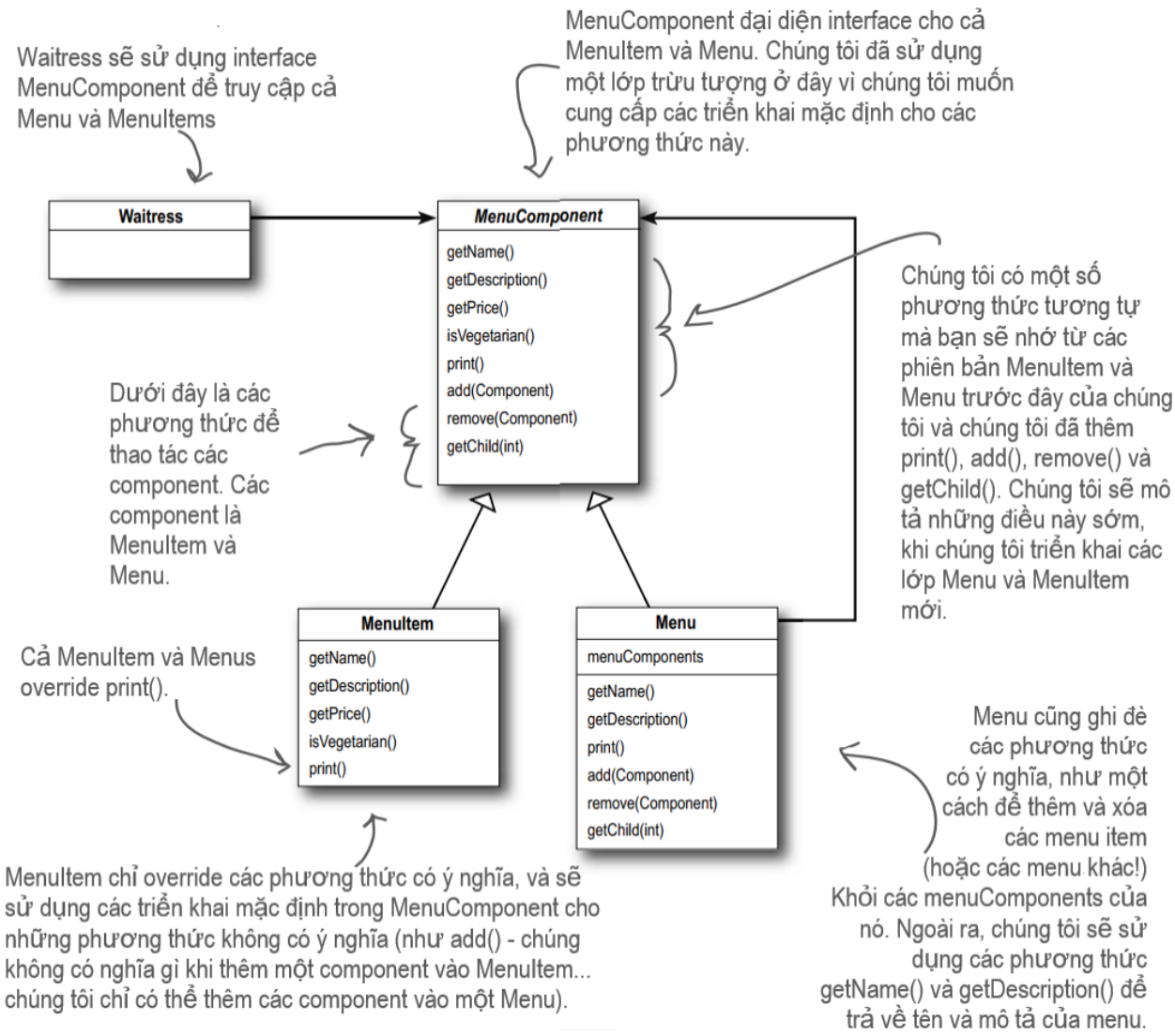
Đáp: Hãy nhớ rằng, chúng tôi đã thực hiện một cách tiếp cận mới. Chúng tôi sẽ thực hiện lại các menu với một giải pháp mới: Composite Pattern. Vì vậy, đừng tìm kiếm một số chuyển đổi từ một Iterator (https://toihocdesignpattern.com/chuong-9-iterator-pattern-va-composite-pattern-p1-iterator-

pattern.html)sang một Composite. “Liên quan đến các iterator” có nghĩa là: hai mẫu này kết hợp với nhau. Bạn sẽ sớm thấy rằng chúng ta có thể sử dụng các iterator (https://toihocdesignpattern.com/chuong-9-iterator-pattern-va-composite-pattern-p1-iterator-pattern.html)theo một vài cách trong việc thực hiện composite.

Thiết kế Menu với Composite pattern

Vì vậy, làm thế nào để chúng tôi áp dụng **Composite Pattern** vào các menu của chúng tôi? Để bắt đầu, chúng ta cần tạo một giao diện component; điều này hoạt động như giao diện chung cho cả menu và các menu item và cho phép chúng ta xử lý chúng đồng nhất. Nói cách khác, chúng ta có thể gọi cùng một phương thức tương tự trên cả các menu và các menu item.

Bây giờ, một số phương thức có thể **không có ý nghĩa** khi gọi trên một menu item hoặc một menu, nhưng chúng ta có thể đối phó với điều đó, và chúng ta sẽ làm trong chốc lát. Nhưng bây giờ, hãy xem qua một bản phác thảo về cách các menu sẽ chuyển thành cấu trúc **Composite Pattern**:



CÀI ĐẶT MENU COMPONENT

Được rồi, chúng tôi sẽ bắt đầu với lớp trừu tượng **MenuComponent**; hãy nhớ rằng, vai trò của menu component là cung cấp interface cho các nút lá và các nút composite. Bây giờ bạn có thể hỏi, "không phải **MenuComponent** đóng hai vai trò sao?", có lẽ nó cũng sẽ như vậy và chúng tôi sẽ quay lại chỗ này bằng một quy tắc thiết kế.

Tuy nhiên, hiện tại chúng tôi sẽ cung cấp một triển khai mặc định của các phương thức để nếu MenuItem (lá) hoặc Menu (composite) không muốn thực hiện một số phương thức (ví dụ nút lá sẽ không cần phương thức **getChild()**) chúng có thể cài đặt cho chúng một số hành vi mặc định.

Tất cả các component phải implement giao diện **MenuComponent**; tuy nhiên, vì các node và leaf có vai trò khác nhau, chúng tôi không thể cài đặt cách triển khai mặc định cho mỗi phương thức đều có ý nghĩa. Đôi khi điều tốt nhất bạn có thể làm là ném runtime exception.

Vì một số phương thức chỉ có ý nghĩa đối với **MenuItem** và một số phương thức chỉ có ý nghĩa đối với **Menu**, nên việc triển khai mặc định sẽ là **UnsupportedOperationException**. Theo cách đó, nếu **MenuItem** hoặc **Menu** không hỗ trợ thao tác, chúng không phải làm gì cả, chúng chỉ có thể kế thừa việc thực hiện mặc định.

MenuComponent cung cấp các implement mặc định cho mọi phương thức

```
public abstract class MenuComponent {

    public void add(MenuComponent menuComponent) {
        throw new UnsupportedOperationException();
    }
    public void remove(MenuComponent menuComponent) {
        throw new UnsupportedOperationException();
    }
    public MenuComponent getChild(int i) {
        throw new UnsupportedOperationException();
    }

    public String getName() {
        throw new UnsupportedOperationException();
    }
    public String getDescription() {
        throw new UnsupportedOperationException();
    }
    public double getPrice() {
        throw new UnsupportedOperationException();
    }
    public boolean isVegetarian() {
        throw new UnsupportedOperationException();
    }

    public void print() {
        throw new UnsupportedOperationException();
    }
}
```

Chúng tôi đã nhóm lại với nhau các phương thức"composite" - đó là các phương thức để thêm, xóa và lấy MenuComponents.

Dưới đây là các phương thức "operation"; những cái này được sử dụng bởi MenuItems. Hóa ra chúng ta cũng có thể sử dụng một vài trong số chúng trong Menu, như bạn sẽ thấy trong một vài trang khi chúng tôi hiển thị Menu code.

print() là một phương thức hoạt động trên cây mà cả Menu và MenuItems của chúng tôi sẽ implement, nhưng chúng tôi cung cấp một hoạt động mặc định ở đây.

CÀI ĐẶT MENU ITEM

Được rồi, hãy thử viết cho lớp **MenuItem**. Hãy nhớ rằng, đây là lớp lá trong Composite diagram và nó thực hiện hành vi của các element của composite.

```
public class MenuItem extends MenuComponent {
    String name;
    String description;
    boolean vegetarian;
    double price;

    public MenuItem(String name,
                    String description,
                    boolean vegetarian,
                    double price)
    {
        this.name = name;
        this.description = description;
        this.vegetarian = vegetarian;
        this.price = price;
    }

    public String getName() {
        return name;
    }

    public String getDescription() {
        return description;
    }

    public double getPrice() {
        return price;
    }

    public boolean isVegetarian() {
        return vegetarian;
    }

    public void print() {
        System.out.print("  " + getName());
        if (isVegetarian()) {
            System.out.print("(v)");
        }
        System.out.println(", " + getPrice());
        System.out.println("     -- " + getDescription());
    }
}
```

Đầu tiên chúng ta cần extend giao diện MenuComponent.

Hàm constructor chỉ lấy tên, mô tả, v.v. và giữ một tham chiếu đến tất cả chúng. Điều này là khá giống như thực hiện menu item cũ của chúng tôi.

Ở đây, các phương thức getter của chúng tôi - giống như cách thực hiện trước đây của chúng tôi.

Điều này khác với việc thực hiện trước đó. Ở đây, chúng tôi đã override phương thức print() trong lớp MenuComponent. Đối với MenuItem phương thức này in ra mục nhập menu hoàn chỉnh: tên, mô tả, giá cả và liệu nó có hay không.

CÀI ĐẶT COMPOSITE MENU

Bây giờ chúng ta có MenuItem, chúng ta chỉ cần lớp composite mà chúng ta gọi là Menu. Hãy nhớ rằng, lớp composite có thể chứa MenuItem (leaf) hoặc các Menu (node) khác. Có một vài phương thức từ MenuComponent, lớp này không triển khai: **getprice()** và **isVegetarian()**, vì những phương thức này không có nhiều ý nghĩa đối với Menu.

Menu cũng là một MenuComponent, giống như MenuItem

Menu có thể có bất kỳ số lượng con kiểu MenuComponent nào, chúng tôi sẽ sử dụng một ArrayList nội bộ để chứa những thứ này.

```
public class Menu extends MenuComponent {
    ArrayList menuComponents = new ArrayList();
    String name;
    String description;

    public Menu(String name, String description) {
        this.name = name;
        this.description = description;
    }

    public void add(MenuComponent menuComponent) {
        menuComponents.add(menuComponent);
    }

    public void remove(MenuComponent menuComponent) {
        menuComponents.remove(menuComponent);
    }

    public MenuComponent getChild(int i) {
        return (MenuComponent)menuComponents.get(i);
    }

    public String getName() {
        return name;
    }

    public String getDescription() {
        return description;
    }

    public void print() {
        System.out.print("\n" + getName());
        System.out.println(", " + getDescription());
        System.out.println("---------------------");
    }
}
```

Điều này khác với cách triển khai cũ của chúng tôi: chúng tôi sẽ cung cấp cho mỗi Menu một tên và một mô tả. Trước đây, chúng tôi chỉ dựa vào việc có các tên lớp khác nhau cho mỗi menu.

Ở đây, cách bạn thêm MenuItems hoặc các Menu khác vào Menu. Vì cả MenuItems và Menu đều là MenuComponents, chúng tôi chỉ cần một phương thức để thực hiện cả hai. Bạn cũng có thể xóa MenuComponent hoặc lấy MenuComponent.

Dưới đây là các phương thức getter để có được tên và mô tả.

Lưu ý, chúng tôi không override getprice() hoặc isVegetarian() bởi vì các phương thức đó không có ý nghĩa đối với Menu (mặc dù bạn có thể cho rằng isVegetarian() có thể có ý nghĩa). Nếu ai đó cố gắng gọi các phương thức đó trên Menu, họ sẽ nhận được UnsupportedOperationException.

Để in Menu, chúng tôi in tên và mô tả cho Menu.



Nắm bắt tuyệt vời. Vì menu là một composite và nó chứa cả Menu Item và các Menu khác, nên phương thức **print()** của nó sẽ in mọi thứ mà nó chứa. Nếu không như vậy, chúng tôi không phải duyệt toàn bộ composite và tự in từng mục. Điều đó sẽ làm sai mục đích của việc có một cấu trúc composite.

Như bạn sẽ thấy, việc triển khai **print()** chính xác rất dễ dàng vì chúng ta có thể dựa vào từng component để có thể tự in. Kiểm tra nó:

SỬA PHƯƠNG THỨC PRINT()

```
public class Menu extends MenuComponent {
    ArrayList menuComponents = new ArrayList();
    String name;
    String description;

    // constructor code here

    // other methods here

    public void print() {
        System.out.print("\n" + getName());
        System.out.println(", " + getDescription());
        System.out.println("---------------------");

        Iterator iterator = menuComponents.iterator();
        while (iterator.hasNext()) {
            MenuComponent menuComponent =
                (MenuComponent)iterator.next();
            menuComponent.print();
        }
    }
}
```

Tất cả những gì chúng ta cần làm là thay đổi phương thức print() để làm cho nó in không chỉ thông tin về Menu này, mà tất cả các component của Menu: Menu khác và Menu Item.

Nhìn kìa! Chúng ta có thể sử dụng một Iterator. Chúng tôi sử dụng nó để lặp qua tất cả các component của Menu... chúng có thể là các Menu khác hoặc chúng có thể là MenuItems. Vì cả Menu và MenuItems đều implement print(), chúng tôi chỉ cần gọi print() và phần còn lại tùy thuộc vào chúng.

LƯU Ý: Nếu trong suốt iteration này, chúng ta bắt gặp một đối tượng Menu khác, phương thức print() của nó sẽ bắt đầu một lần lặp khác, v.v.

Sẵn sàng cho một test drive...

Đã đến lúc chúng tôi lấy code này để chạy thử, nhưng chúng tôi cần cập nhật code Waitress trước khi thực hiện – sau tất cả, Waitress là client của đoạn code:

```
public class Waitress {
    MenuComponent allMenus;

    public Waitress(MenuComponent allMenus) {
        this.allMenus = allMenus;
    }

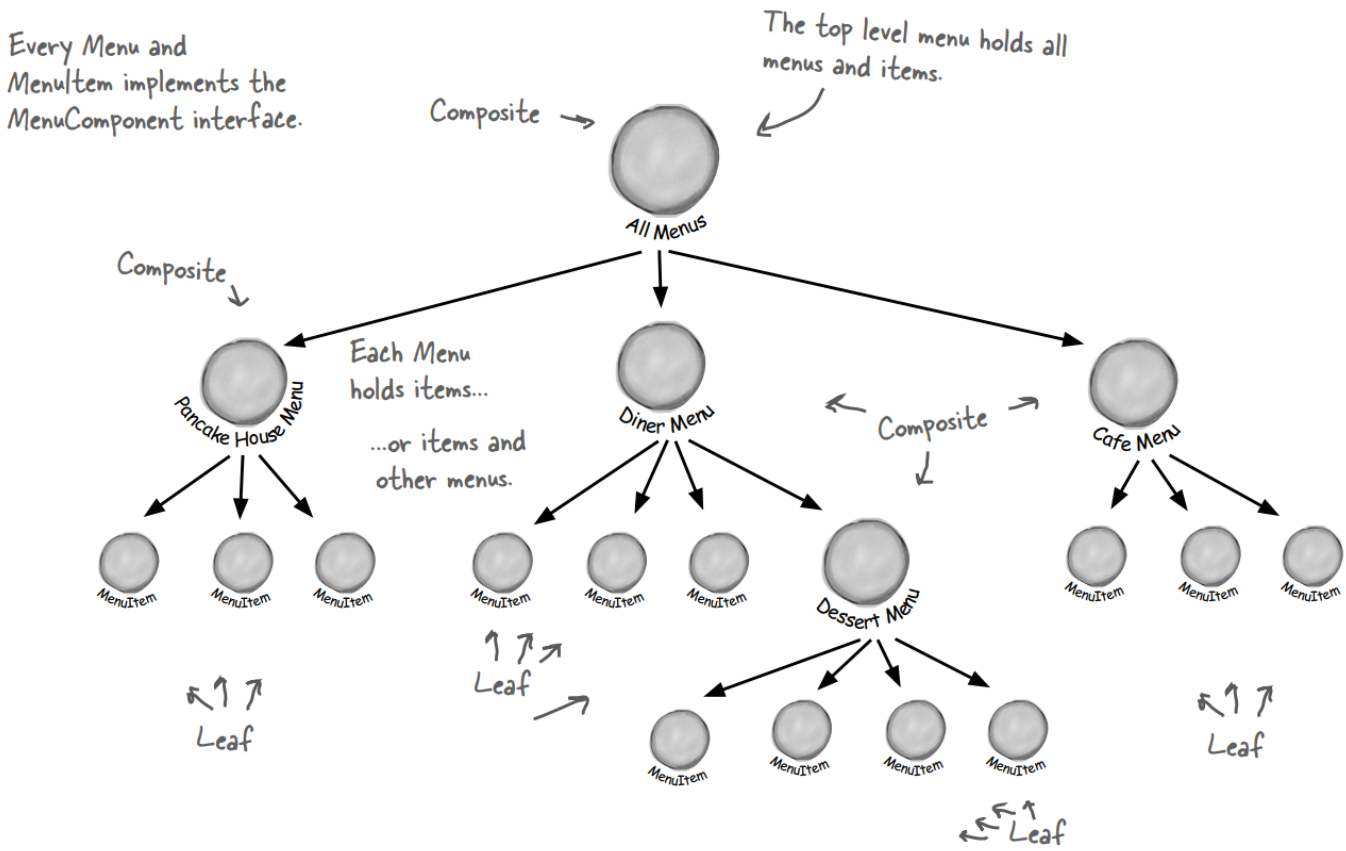
    public void printMenu() {
        allMenus.print();
    }
}
```

Vâng code Waitress thực sự đơn giản. Bây giờ chúng tôi chỉ đưa cho cô ấy component menu cấp cao nhất, một component chứa tất cả các menu khác. Chúng tôi đã gọi đó là allMenus.

Tất cả những gì cô ấy phải làm để in toàn bộ hệ thống phân cấp menu - tất cả các menu và tất cả các menu item - là gọi print() trên menu cấp cao nhất.

Chúng tôi sẽ có một Waitress hạnh phúc.

Được rồi, một điều cuối cùng trước khi chúng tôi viết test drive của chúng tôi. Hãy có ý tưởng về việc component menu sẽ trông như thế nào trong thời gian chạy runtime:



BÂY GIỜ CHO TEST DRIVE...

Được rồi, bây giờ chúng tôi chỉ cần một test drive. Không giống như phiên bản trước của chúng tôi, chúng tôi sẽ xử lý tất cả việc tạo menu trong test drive. Chúng tôi có thể yêu cầu mỗi đầu bếp cung cấp cho chúng tôi menu mới của anh ấy, nhưng hãy thử nghiệm tất cả trước. Đây là code:

```
public class MenuTestDrive {
    public static void main(String args[]) {
        MenuComponent pancakeHouseMenu =
            new Menu("PANCAKE HOUSE MENU", "Breakfast");
        MenuComponent dinerMenu =
            new Menu("DINER MENU", "Lunch");
        MenuComponent cafeMenu =
            new Menu("CAFE MENU", "Dinner");
        MenuComponent dessertMenu =
            new Menu("DESSERT MENU", "Dessert of course!");

        MenuComponent allMenus = new Menu("ALL MENUS", "All menus combined");

        allMenus.add(pancakeHouseMenu);
        allMenus.add(dinerMenu);
        allMenus.add(cafeMenu);

        // add menu items here

        dinerMenu.add(new MenuItem(
            "Pasta",
            "Spaghetti with Marinara Sauce, and a slice of sourdough bread",
            true,
            3.89));

        dinerMenu.add(dessertMenu);

        dessertMenu.add(new MenuItem(
            "Apple Pie",
            "Apple pie with a flakey crust, topped with vanilla icecream",
            true,
            1.59));

        // add more menu items here

        Waitress waitress = new Waitress(allMenus);

        waitress.printMenu();
    }
}
```

Trước tiên, hãy tạo ra tất cả các đối tượng menu.

Chúng tôi cũng cần hai menu cấp cao nhất bây giờ mà chúng tôi sẽ đặt tên cho allMenus.

Chúng tôi đã sử dụng phương thức Composite add() để thêm từng menu vào menu cấp cao nhất, allMenus.

Bây giờ chúng ta cần thêm tất cả các menu item, ở đây, một ví dụ, phần còn lại, hãy xem sourcecode hoàn chỉnh

Và chúng tôi cũng có thể thêm một menu vào menu. Tất cả các dinerMenu quan tâm là tất cả mọi thứ nó chứa, cho dù đó là một menu item hay menu, đều là một MenuComponent.

Thêm một số bánh táo vào thực đơn tráng miệng...

Khi chúng tôi xây dựng toàn bộ hệ thống phân cấp menu của mình, chúng tôi giao toàn bộ cho Waitress, và như bạn đã thấy, nó dễ dàng như một chiếc bánh táo để Waitress in ra.

SẴN SÀNG CHO MỘT TEST DRIVE...

(LƯU Ý: kết quả này dựa trên sourcecode hoàn chỉnh)

```
File Edit Window Help GreenEggs&Spam
% java MenuTestDrive

ALL MENUS, All menus combined
---------------------

PANCAKE HOUSE MENU, Breakfast
---------------------
  K&B's Pancake Breakfast(v), 2.99
     -- Pancakes with scrambled eggs, and toast
  Regular Pancake Breakfast, 2.99
     -- Pancakes with fried eggs, sausage
  Blueberry Pancakes(v), 3.49
     -- Pancakes made with fresh blueberries, and blueberry syrup
  Waffles(v), 3.59
     -- Waffles, with your choice of blueberries or strawberries

DINER MENU, Lunch
---------------------
  Vegetarian BLT(v), 2.99
     -- (Fakin') Bacon with lettuce & tomato on whole wheat
  BLT, 2.99
     -- Bacon with lettuce & tomato on whole wheat
  Soup of the day, 3.29
     -- A bowl of the soup of the day, with a side of potato salad
  Hotdog, 3.05
     -- A hot dog, with saurkraut, relish, onions, topped with cheese
  Steamed Veggies and Brown Rice(v), 3.99
     -- Steamed vegetables over brown rice
  Pasta(v), 3.89
     -- Spaghetti with Marinara Sauce, and a slice of sourdough bread

DESSERT MENU, Dessert of course!
---------------------
  Apple Pie(v), 1.59
     -- Apple pie with a flakey crust, topped with vanilla icecream
  Cheesecake(v), 1.99
     -- Creamy New York cheesecake, with a chocolate graham crust
  Sorbet(v), 1.89
     -- A scoop of raspberry and a scoop of lime

CAFE MENU, Dinner
---------------------
  Veggie Burger and Air Fries(v), 3.99
     -- Veggie burger on a whole wheat bun, lettuce, tomato, and fries
  Soup of the day, 3.69
     -- A cup of the soup of the day, with a side salad
  Burrito(v), 4.29
     -- A large burrito, with whole pinto beans, salsa, guacamole
%
```

Here's all our menus... we printed all this just by calling print() on the top level menu

The new dessert menu is printed when we are printing all the Diner menu components



Có một số sự thật cho quan sát đó. Chúng ta có thể nói rằng Composite Pattern áp dụng nguyên tắc **Single Responsibility** và biến đổi nó để trong suốt (*transparency*). *Transparency* gì? Chà, bằng cách cho phép Component interface chứa các hoạt động quản lý node và leaf, một client có thể xử lý node và leaf một cách tương tự; vì vậy một phần tử có thể là nút composite hay nút lá đều trở nên "trong suốt" đối với client.

Bây giờ chúng ta có cả hai loại hoạt động trong lớp Component, chúng ta mất một chút an toàn vì client có thể cố gắng làm điều gì đó không phù hợp hoặc vô nghĩa trên một Component (như thử thêm menu vào một menu item). Đây là một quyết định thiết kế; chúng ta có thể đưa thiết kế theo hướng khác và tách các trách nhiệm thành các interface. Điều này sẽ làm cho thiết kế của chúng ta an toàn, theo nghĩa là bất kỳ lệnh gọi không phù hợp nào đối với các phần tử sẽ bị catch trong thời gian biên dịch hoặc runtime, nhưng chúng tôi mất tính *transparency* và code của chúng tôi sẽ phải sử dụng các điều kiện và toán tử **instanceof**.

Vì vậy, để trở lại câu hỏi của bạn, đây là một trường hợp đánh đổi kinh điển. Chúng tôi được hướng dẫn bởi các nguyên tắc thiết kế, nhưng chúng tôi luôn cần phải xem xét ảnh hưởng của chúng đối với các thiết kế. Đôi khi chúng tôi cố tình làm mọi thứ theo cách dường như vi phạm nguyên tắc.

Tuy nhiên, trong một số trường hợp, đây là vấn đề về quan điểm; chẳng hạn, sẽ không chính xác khi có các phương thức quản lý nút con trong các nút lá (nút lá sẽ không cần **add()**, **remove()** và **getChild()**), nhưng sau đó, bạn cũng có thể thay đổi quan điểm của mình và xem một nút lá như một node không có nút con và các phương thức **add()**, **remove()** và **getChild()** sẽ trở nên phù hợp.

Nhớ lại mẫu Iterator và kết hợp chúng với Composite Pattern

Chúng tôi đã hứa với bạn rằng chúng tôi sẽ chỉ cho bạn cách sử dụng Iterator (https://toihocdesignpattern.com/chuong-9-iterator-pattern-va-composite-pattern-p1-iterator-pattern.html)với một Composite. Bạn biết rằng chúng tôi đã sử dụng Iterator (https://toihocdesignpattern.com/chuong-9-iterator-pattern-va-composite-pattern-p1-iterator-pattern.html)trong cài đặt của phương thức **print()**, nhưng chúng tôi cũng có thể cho phép Waitress duyệt toàn bộ composite nếu cần, ví dụ, nếu cô ấy muốn duyệt qua toàn bộ menu và lấy tất cả vegetarian item ra.

Để cài đặt một Composite iterator, hãy thêm một phương thức **createrterator()** trong mọi component. Chúng tôi sẽ bắt đầu với lớp trừu tượng **MenuComponent**:



Bây giờ chúng ta cần thực hiện phương thức này trong các lớp **Menu** và **MenuItem**:

```
public class Menu extends MenuComponent {
    Iterator iterator = null;
    // other code here doesn't change

    public Iterator createIterator() {
        if (iterator == null) {
            iterator = new CompositeIterator(menuComponents.iterator());
        }
        return iterator;
    }
}


public class MenuItem extends MenuComponent {

    // other code here doesn't change

    public Iterator createIterator() {
        return new NullIterator();
    }

}
```

Chúng tôi chỉ cần một iterator cho mỗi Menu.

Ở đây, chúng tôi sử dụng một iterator mới gọi là CompositeIterator. Nó biết làm thế nào để lặp đi lặp lại trên bất kỳ composite.

Chúng tôi đưa iterator của composite hiện tại vào

Whoa! Cái gì mà NullIterator này? Bạn sẽ thấy trong hai trang sau.

Composite Iterator

CompositeIterator là một iterator HẤP DẪN. Công việc của nó là duyệt phần tử trên các **MenuItem** trong component và đảm bảo tất cả các sub-Menu (và sub-sub-Menu, v.v.) cũng được duyệt.

Đây là code. Xem ra, ở đây không có quá nhiều code, nhưng nó có thể là một chút thay đổi về suy nghĩ. Chỉ cần lặp lại với chính mình khi bạn đọc qua nó "đệ quy là bạn của tôi, đệ quy là bạn của tôi."

```
import java.util.*;

public class CompositeIterator implements Iterator {
    Stack stack = new Stack();

    public CompositeIterator(Iterator iterator) {
        stack.push(iterator);
    }

    public Object next() {
        if (hasNext()) {
            Iterator iterator = (Iterator) stack.peek();
            MenuComponent component = (MenuComponent) iterator.next();
            if (component instanceof Menu) {
                stack.push(component.createIterator());
            }
            return component;
        } else {
            return null;
        }
    }

    public boolean hasNext() {
        if (stack.empty()) {
            return false;
        } else {
            Iterator iterator = (Iterator) stack.peek();
            if (!iterator.hasNext()) {
                stack.pop();
                return hasNext();
            } else {
                return true;
            }
        }
    }

    public void remove() {
        throw new UnsupportedOperationException();
    }
}
```

Giống như tất cả các iterator, chúng tôi đang triển khai giao diện java.util.Iterator

Iterator của composite cấp cao nhất mà chúng tôi sẽ lặp được đưa vào. Chúng tôi đưa nó vào một cấu trúc dữ liệu Stack.

Được rồi, khi client muốn lấy phần tử tiếp theo, trước tiên chúng tôi đảm bảo có phần tử bằng cách gọi hasNext() ...

Nếu có một phần tử tiếp theo, chúng ta sẽ đưa phần tử hiện tại ra khỏi stack và lấy phần tử tiếp theo của nó.

Nếu phần tử đó là một menu, chúng ta có một composite khác cần được bao gồm trong lần lặp, vì vậy chúng tôi đưa nó vào stack.
Trong cả hai trường hợp, chúng tôi return component

Để xem có phần tử tiếp theo không, chúng tôi kiểm tra xem stack có rỗng không; nếu vậy thì không có.
Mặt khác, chúng ta lấy iterator ra khỏi đỉnh stack và xem nó có phần tử tiếp theo không. Nếu không, chúng ta sẽ Pop nó ra khỏi ngăn xếp và gọi đệ quy hasNext().

Nếu không, có một phần tử tiếp theo và chúng tôi return true.

Chúng không hỗ trợ remove



Khi chúng ta viết phương thức **print()** trong lớp **MenuComponent**, chúng ta đã sử dụng một iterator để duyệt qua từng phần tử trong component và nếu phần tử đó là **Menu** (chứ không phải là **MenuItem**), thì chúng ta gọi đệ quy phương thức **print()** để xử lý nó. Nói cách khác, **MenuComponent** đã tự xử lý việc duyệt phần tử bên trong nó.

Với code này, chúng tôi đang triển khai một *external* iterator để có nhiều thứ hơn để theo dõi. Đối với người mới bắt đầu, một *external* iterator phải duy trì vị trí của nó trong vòng lặp để client bên ngoài có thể điều khiển vòng lặp bằng cách gọi **hasNext()** và **next()**. Trong trường hợp này, code của chúng tôi cần duy trì vị trí trên một composite. Đó là lý do tại sao chúng tôi sử dụng Stack để duy trì vị trí khi chúng tôi di chuyển lên và xuống hệ thống phân cấp composite.

Bài tập

Vẽ sơ đồ của **Menu** và **MenuItem**. Sau đó, giả vờ bạn là **CompositeIterator** và công việc của bạn là xử lý các cuộc gọi đến **hasNext()** và **next()**. Theo dõi cách **CompositeIterator** duyệt qua cấu trúc khi code này được thực thi:

```
public void testCompositeIterator(MenuComponent component) {
       CompositeIterator iterator = new CompositeIterator(component.iterator);

       while(iterator.hasNext()) {
              MenuComponent component = iterator.next();
       }
}
```

Null Iterator

Được rồi, bây giờ Null Iterator này là gì? Hãy suy nghĩ về nó theo cách này: một MenuItem không có gì để duyệt, phải không? Vậy làm thế nào để chúng ta xử lý việc thực hiện phương thức **createIterator()** của nó? Vâng, chúng tôi có hai sự lựa chọn:

Lựa chọn 1: Return null

Chúng tôi có thể return null từ **createIterator()**, nhưng sau đó chúng tôi sẽ cần code điều kiện (if) trong client để xem null có được return hay không.

Lựa chọn 2: Return một iterator luôn return false khi hasNext() được gọi

Đây dường như là một kế hoạch tốt hơn. Chúng tôi vẫn có return về một iterator, nhưng client không phải lo lắng về việc có bao giờ null được return hay không. Thực tế, chúng tôi đã tạo ra một iterator là một "lệnh vô tác" (no-operation – toán tử không thực hiện thao tác)".

Sự lựa chọn thứ hai chắc chắn có vẻ tốt hơn. Hãy gọi nó là **NullIterator** và thực hiện nó.

```
import java.util.Iterator;

public class NullIterator implements Iterator {

    public Object next() {
        return null;
    }

    public boolean hasNext() {
        return false;
    }

    public void remove() {
        throw new UnsupportedOperationException();
    }
}
```

Đây là Iterator "lười biếng" nhất mà bạn đã từng thấy.

Khi next() được gọi, chúng ta return null.

Quan trọng nhất là khi hasNext() được gọi, chúng ta luôn return false.

Và **NullIterator** sẽ không hỗ trợ remove().

Cho tôi thực đơn chay!

Bây giờ chúng tôi đã có một cách để duyệt trên mọi item của Menu. Hãy đưa cho Waitress của chúng ta một method có thể cho chúng ta biết chính xác món nào là món chay (vegetarian).

```
public class Waitress {
    MenuComponent allMenus;

    public Waitress(MenuComponent allMenus) {
        this.allMenus = allMenus;
    }

    public void printMenu() {
        allMenus.print();
    }

    public void printVegetarianMenu() {
        Iterator iterator = allMenus.createIterator();
        System.out.println("\nVEGETARIAN MENU\n----");
        while (iterator.hasNext()) {
            MenuComponent menuComponent =
                    (MenuComponent)iterator.next();
            try {
                if (menuComponent.isVegetarian()) {
                    menuComponent.print();
                }
            } catch (UnsupportedOperationException e) {}
        }
    }
}
```

Method printVegetarianMenu() lấy composite của allMenu và lấy iterator của nó. Đó sẽ là CompositeIterator của chúng tôi.

Lặp lại qua mọi phần tử của composite.

Gọi Phương thức isVegetarian() trên mỗi phần tử và nếu true, chúng ta gọi phương thức print() của nó.

print() chỉ được gọi trên MenuItems, không bao giờ gọi trên composite. Bạn có biết tại sao không?

Chúng tôi đã triển khai isVegetarian() trên Menu để luôn ném exception. Nếu điều đó xảy ra, chúng ta bắt ngoại lệ, nhưng vẫn tiếp tục với phép lặp của chúng ta.

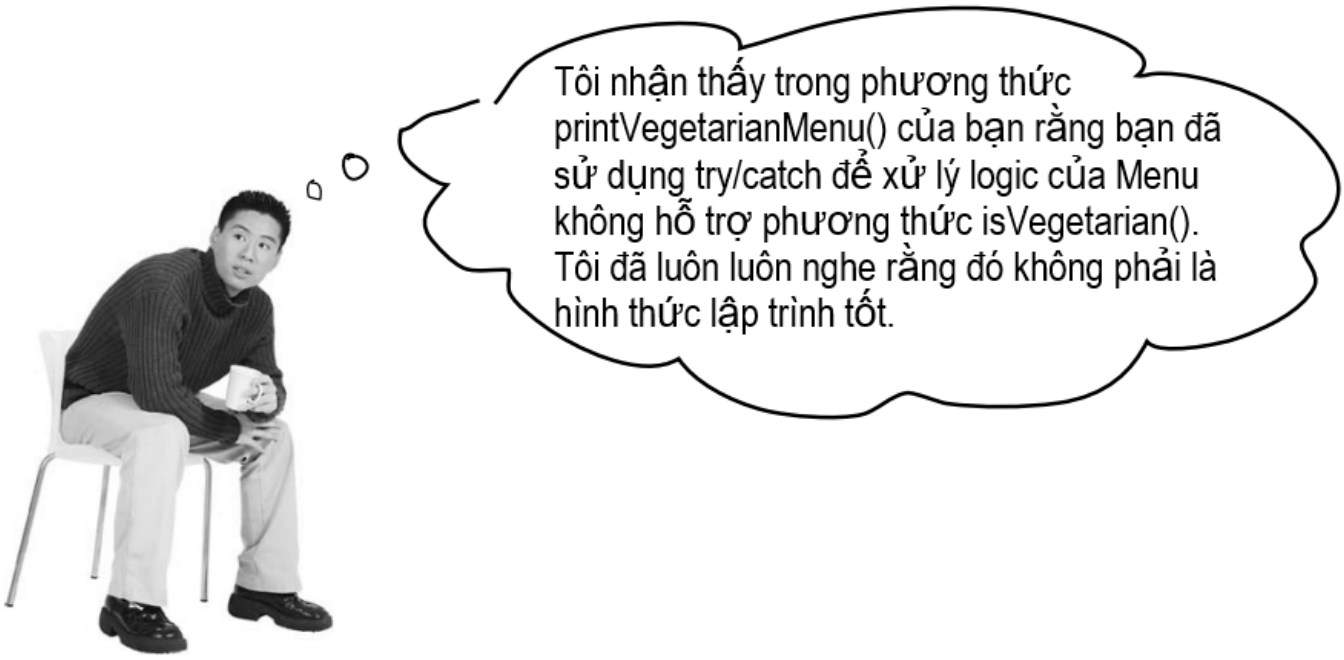
Kết hợp Iterator & Composite Pattern với nhau...

Ôi trời! Đó là một nỗ lực phát triển để đưa code của chúng tôi đến thời điểm này. Bây giờ chúng tôi đã có một cấu trúc menu chung sẽ tồn tại lâu dài trong đế chế Diner đang phát triển. Bây giờ đã đến lúc ngồi lại và gọi một ít thức ăn chay (Vegetarian food):

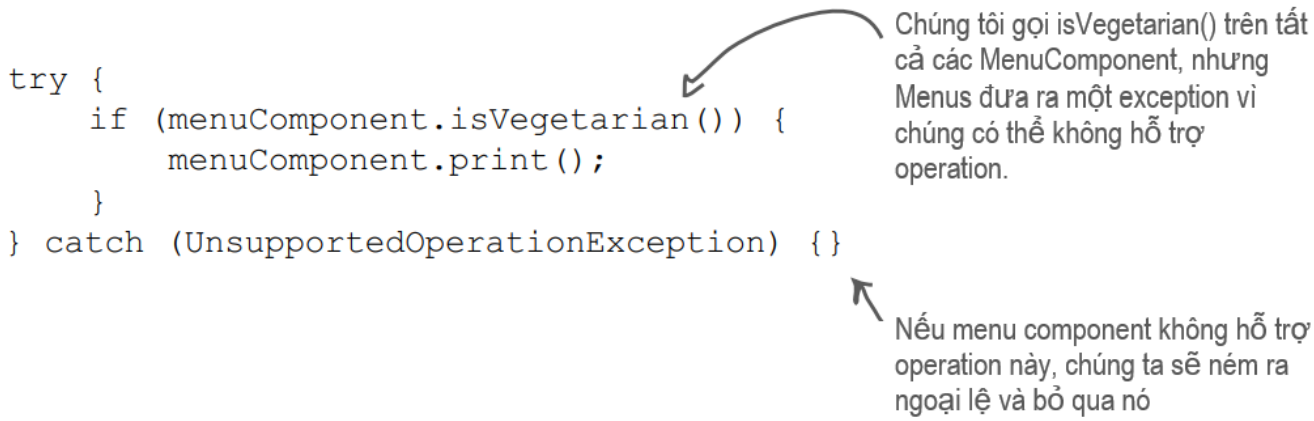
```
File Edit Window Help HaveUhuggedYurIteratorToday?
% java MenuTestDrive

VEGETARIAN MENU
----
  K&B's Pancake Breakfast(v), 2.99
     -- Pancakes with scrambled eggs, and toast
  Blueberry Pancakes(v), 3.49
     -- Pancakes made with fresh blueberries, and blueberry syrup
  Waffles(v), 3.59
     -- Waffles, with your choice of blueberries or strawberries
  Vegetarian BLT(v), 2.99
     -- (Fakin') Bacon with lettuce & tomato on whole wheat
  Steamed Veggies and Brown Rice(v), 3.99
     -- Steamed vegetables over brown rice
  Pasta(v), 3.89
     -- Spaghetti with Marinara Sauce, and a slice of sourdough bread
  Apple Pie(v), 1.59
     -- Apple pie with a flakey crust, topped with vanilla icecream
  Cheesecake(v), 1.99
     -- Creamy New York cheesecake, with a chocolate graham crust
  Sorbet(v), 1.89
     -- A scoop of raspberry and a scoop of lime
  Apple Pie(v), 1.59
     -- Apple pie with a flakey crust, topped with vanilla icecream
  Cheesecake(v), 1.99
     -- Creamy New York cheesecake, with a chocolate graham crust
  Sorbet(v), 1.89
     -- A scoop of raspberry and a scoop of lime
  Veggie Burger and Air Fries(v), 3.99
     -- Veggie burger on a whole wheat bun, lettuce, tomato, and fries
  Burrito(v), 4.29
     -- A large burrito, with whole pinto beans, salsa, guacamole
%
```

The Vegetarian Menu consists of the vegetarian items from every menu.



Hãy cùng xem những gì bạn đang nói:



Nói chung chúng tôi đồng ý; **try/catch** là để xử lý lỗi, không phải dành cho logic chương trình. Các lựa chọn khác của chúng ta là gì? Chúng ta có thể kiểm tra loại menu component trong runtime với **instanceof** để đảm bảo rằng nó là **MenuItem** trước khi gọi đến **isVegetarian()**. Nhưng trong quá trình đó, chúng tôi mất đi sự trong suốt (*transparency – client bên ngoài nhìn thấy Menu hay MenuItem như nhau vì chúng đều là MenuComponent*) vì chúng tôi sẽ đối xử với Menu và MenuItem một cách thống nhất.

Chúng ta cũng có thể thay đổi **isVegetarian()** trong Menu để nó return false. Điều này cung cấp một giải pháp đơn giản và chúng vẫn *transparency*.

Trong giải pháp của chúng tôi, chúng tôi làm mọi thứ rõ ràng hơn (không còn *transparency* nữa): chúng tôi muốn biết rằng: đây là một phương thức có được hỗ trợ trên Menu hay không (khác với cách làm return false trong **isVegetarian()**).

Phỏng vấn tuần này: Composite Pattern nói về các vấn đề implementation

HeadFirst: Tối nay, chúng tôi ở đây nói chuyện với Composite Pattern. Tại sao anh không nói với chúng tôi một chút về bản thân, Composite?

Composite: Chắc chắn rồi… Tôi là một mẫu để sử dụng khi bạn có tập hợp các đối tượng có mối quan hệ toàn bộ (whole-part relationship) và bạn mong muốn có thể xử lý các đối tượng đó một cách thống nhất theo cùng một cách.

HeadFirst: Được rồi, hãy dừng lại một chút ngay tại đây …"mối quan hệ toàn bộ" là gì?

Composite: Tưởng tượng một giao diện người dùng đồ họa (GUI); ở đó, bạn sẽ thường tìm thấy một thành phần cấp cao nhất như Frame hoặc Panel, có chứa các thành phần khác bên trong, như menu, text pane, scrollbar và button. Vì vậy, GUI của bạn bao gồm nhiều phần, nhưng khi bạn hiển thị nó, bạn

thường nghĩ về nó như một tổng thể. Bạn nói với thành phần cấp cao nhất (top level) sẽ hiển thị và dựa vào nó để hiển thị tất cả các thành phần bên trong của nó. Chúng tôi gọi các thành phần có chứa các thành phần khác bên trong là **Composite** và các thành phần không chứa các thành phần khác là **Leaf**.

HeadFirst: Đó có phải là ý của anh khi đối xử thống nhất với các đối tượng? Có một phương thức chung anh có thể gọi cả trên composite và leaf phải không?

Composite: Phải. Tôi có thể nói với một composite object hoặc một leaf object khi muốn hiển thị và nó sẽ làm đúng. Composite object sẽ hiển thị bằng cách thông báo cho tất cả các thành phần bên trong của nó hiển thị.

HeadFirst: Điều đó ngụ ý rằng mọi đối tượng đều có cùng interface. Điều gì nếu anh có các đối tượng trong composite của anh, nhưng chúng lại làm những việc khác nhau?

Composite: Vâng, để composite hoạt động một cách *transparent* đối với client, bạn phải triển khai cùng một interface cho tất cả các đối tượng trong composite, nếu không, client phải lo lắng về việc đối tượng nào đang implement interface nào, loại nào làm những gì. Rõ ràng điều đó có nghĩa là đôi khi bạn có thể có các đối tượng mà với vài phương thức được gọi, nhưng không có nghĩa.

HeadFirst: Vậy làm thế nào để anh xử lý điều đó?

Composite: Vâng, có một vài cách để xử lý nó; đôi khi bạn không thể làm gì hoặc return null hoặc false – bất cứ điều gì có ý nghĩa trong ứng dụng của bạn. Những lần khác, bạn sẽ muốn chủ động hơn và đưa ra một exception. Tất nhiên, sau đó client phải sẵn sàng thực hiện một công việc nhỏ là bắt exception đó và đảm bảo rằng không sinh ra lỗi không mong đợi.

HeadFirst: Nhưng nếu client không biết loại đối tượng mà họ đang xử lý, làm sao họ biết cuộc gọi nào sẽ thực hiện mà không kiểm tra loại đối tượng?

Composite: Nếu bạn có một chút sáng tạo, bạn có thể cấu trúc các phương thức của mình để các default implementation thực hiện điều gì đó có ý nghĩa. Chẳng hạn, nếu client đang gọi **getChild()**, thì trên composite, điều này có ý nghĩa. Và nó cũng có ý nghĩa trên một nút lá, nếu bạn nghĩ về nút lá như một composite không có nút con.

HeadFirst: Ah … thông minh. Nhưng, tôi đã nghe một số client rất lo lắng về vấn đề này, họ yêu cầu các interface riêng cho các đối tượng khác nhau vì vậy chúng không được phép thực hiện các cuộc gọi phương thức vô nghĩa. Có phải đó vẫn là Composite Pattern?

Composite: Vâng. Nó có phiên bản an toàn hơn nhiều cho Composite Pattern, nhưng nó yêu cầu client kiểm tra loại của mọi đối tượng trước khi thực hiện cuộc gọi để đối tượng có thể được truyền chính xác.

HeadFirst: Hãy cho chúng tôi biết thêm một chút về cách các đối tượng composite và leaf được cấu trúc.

Composite: Thông thường, nó có cấu trúc cây, một loại của thừa kế. Nút gốc là composite cấp cao nhất và tất cả các con của nó là composite hoặc nút lá (leaf).

HeadFirst: Nút con có bao giờ trỏ ngược lại nút cha không?

Composite: Có, một component có thể có một con trỏ tới cha để làm cho việc duyệt qua cấu trúc dễ dàng hơn. Và, nếu bạn có một tham chiếu một nút con, và bạn cần xóa nó, bạn sẽ cần phải có nút cha để loại bỏ nút con đó. Có tham chiếu tới nút cha làm cho điều đó cũng dễ dàng hơn.

HeadFirst: Có rất nhiều thứ để xem xét khi thực hiện. Có vấn đề nào khác chúng ta nên suy nghĩ khi thực hiện Composite Pattern không?

Composite: Thật ra có… một là thứ tự của nút con. Điều gì nếu bạn có một composite cần phải giữ con của nó theo một thứ tự cụ thể? Sau đó, bạn sẽ cần một sơ đồ quản lý tinh vi hơn để thêm và xóa nút con, và bạn sẽ phải cẩn thận về cách bạn duyệt qua hệ thống phân cấp.

HeadFirst: Một điểm hay mà tôi đã nghĩ đến.

Composite: Và bạn đã nghĩ về bộ nhớ đệm (caching) chưa?

HeadFirst: Bộ nhớ đệm?

Composite: Yeah, bộ nhớ đệm. Đôi khi, nếu cấu trúc composite phức tạp hoặc tốn kém để duyệt qua, nó có ích để thực hiện lưu trữ bộ đệm của các nút composite. Chẳng hạn, nếu bạn liên tục duyệt qua một composite và tất cả các phần tử con của nó để tính toán một số kết quả, bạn có thể triển khai bộ đệm lưu trữ kết quả tạm thời để lưu các lần duyệt.

HeadFirst: Chà, có rất nhiều điều về Composite Pattern, nhiều hơn tôi từng nghĩ. Trước khi chúng tôi kết thúc buổi phỏng vấn, thêm một câu hỏi nữa: Anh nghĩ gì về sức mạnh lớn nhất của mình?

Composite: Tôi nghĩ rằng tôi đơn giản hóa cuộc sống cho client của tôi. Các client của tôi không phải lo lắng về việc họ có đang xử lý một đối tượng composite hay một đối tượng lá hay không, vì vậy họ không phải viết các câu lệnh ở khắp mọi nơi để đảm bảo rằng họ gọi đúng phương thức trên đúng đối tượng.

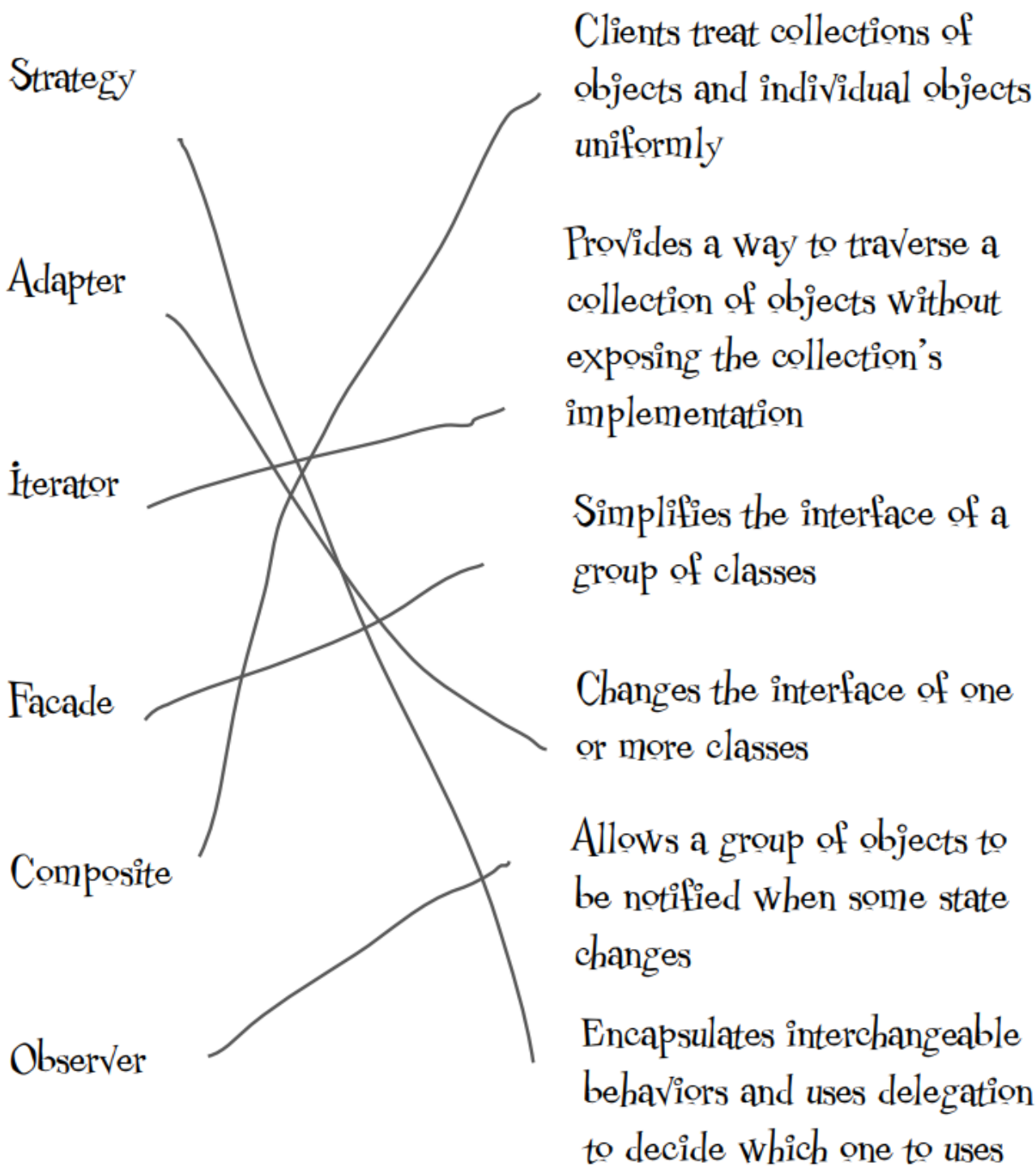
Thông thường, họ chỉ thực hiện một cuộc gọi phương thức và chúng sẽ thực hiện một thao tác trên toàn bộ cấu trúc.

HeadFirst: Nghe có vẻ như là một lợi ích quan trọng. Không có nghi ngờ gì nữa, anh có thể có một pattern hữu ích để thu thập và quản lý các đối tượng. Và với điều đó, chúng tôi đã hết thời gian… Cảm ơn rất nhiều vì đã tham gia với chúng tôi và chúng ta sẽ sớm quay lại với các mẫu khác.

Nối từng mẫu với định nghĩa của nó

Pattern	Description
Strategy	Clients treat collections of objects and individual objects uniformly
Adapter	Provides a way to traverse a collection of objects without exposing the collection's implementation
Iterator	Simplifies the interface of a group of classes
Facade	Changes the interface of one or more classes
Composite	Allows a group of objects to be notified when some state changes
Observer	Encapsulates interchangeable behaviors and uses delegation to decide which one to uses

Đáp án:



Tóm tắt

- Một Iterator cho phép truy cập vào một phần tử tập hợp mà không làm lộ cấu trúc bên trong của nó.
- Một Iterator (https://toihocdesignpattern.com/chuong-9-iterator-pattern-va-composite-pattern-p1-iterator-pattern.html)đảm nhận công việc duyệt phần tử trên một tập hợp và gói nó trong một đối tượng khác.
- Khi sử dụng Iterator, chúng tôi giảm bớt trách nhiệm tổng hợp của các hoạt động hỗ trợ cho việc truyền tải dữ liệu của nó.
- Iterator cung cấp một giao diện chung để duyệt qua các item của tập hợp, cho phép bạn sử dụng đa hình khi viết code sử dụng các item của tập hợp.

- Chúng ta nên cố gắng chỉ giao một trách nhiệm cho mỗi lớp.
- Composite Pattern cung cấp một cấu trúc để chứa cả các leaf object và composite con.
- Composite Pattern cho phép client xử lý composite và các leaf object một cách thống nhất.
- Component là bất kỳ đối tượng nào trong cấu trúc cây của Composite. Các Component có thể là Composite khác hoặc các nút lá.
- Có nhiều sự đánh đổi trong thiết kế khi cài đặt Composite pattern. Bạn cần phải cân bằng sự transparent và sự an toàn với nhu cầu của bạn.